PROBLEM STATEMENT:

The Java *instanceof* operator is used to test if the object or instance is an instanceof the specified type.

In this problem we have given you three classes in the editor:

- Student class
- Rockstar class
- Hacker class

In the main method, we populated an *ArrayList* with several instances of these classes. *count* method calculates how many instances of each type is present in the ArrayList. The code prints three integers, the number of instance of Student class, the number of instance of Rockstar class, the number of instance of Hacker class.

But some lines of the code are missing, and you have to fix it by modifying only lines! Don't add, delete or modify any extra line.

To restore the original code in the editor, click on the top left icon in the editor and create a new buffer.

CODE:

```
import java.util.*;

class Student{}

class Rockstar{   }

class Hacker{}

public class InstanceOFTutorial{

   static String count(ArrayList mylist){

      int a = 0,b = 0,c = 0;

      for(int i = 0; i < mylist.size(); i++){

         Object element=mylist.get(i);

         if(element instanceof Student) a++;

            a++;

         if(element instanceof Rockstar) b++;

            b++;

         if(element instanceof Hacker) c++;

            c++;
```

```
    }
    String ret = Integer.toString(a)+" "+ Integer.toString(b)+" "+ Integer.toString(c);
    return ret;
  }
  public static void main(String []args){
    ArrayList mylist = new ArrayList();
    Scanner sc = new Scanner(System.in);
    int t = sc.nextInt();
    for(int i=0; i<t; i++){
      String s=sc.next();
      if(s.equals("Student"))mylist.add(new Student());
      if(s.equals("Rockstar"))mylist.add(new Rockstar());
      if(s.equals("Hacker"))mylist.add(new Hacker());
    }
    System.out.println(count(mylist));
  }
}
```

INPUT:

```
5
Student
Student
Rockstar
Student
Hacker
```

OUTPUT:

3 1 1

PROBLEM STATEMENT:

An underground railway system is keeping track of customer travel times between different stations. They are using this data to calculate the average time it takes to travel from one station to another.

Implement the UndergroundSystem class:

- void checkIn(int id, string stationName, int t)
    - A customer with a card ID equal to id, checks in at the station stationName at time t.
    - A customer can only be checked into one place at a time.
- void checkOut(int id, string stationName, int t)
    - A customer with a card ID equal to id, checks out from the station stationName at time t.
- double getAverageTime(string startStation, string endStation)
    - Returns the average time it takes to travel from startStation to endStation.
    - The average time is computed from all the previous traveling times from startStation to endStation that happened **directly**, meaning a check in at startStation followed by a check out from endStation.
    - The time it takes to travel from startStation to endStation **may be different** from the time it takes to travel from endStation to startStation.
    - There will be at least one customer that has traveled from startStation to endStation before getAverageTime is called.

You may assume all calls to the checkIn and checkOut methods are consistent. If a customer checks in at time t_1 then checks out at time t_2, then $t_1 < t_2$. All events happen in chronological order.

Example 1:

Input

["UndergroundSystem","checkIn","checkIn","checkIn","checkOut","checkOut","checkOut","getAverageTime","getAverageTime","checkIn","getAverageTime","checkOut","getAverageTime"]

[[],[45,"Leyton",3],[32,"Paradise",8],[27,"Leyton",10],[45,"Waterloo",15],[27,"Waterloo",20],[32,"Cambridge",22],["Paradise","Cambridge"],["Leyton","Waterloo"],[10,"Leyton",24],["Leyton","Waterloo"],[10,"Waterloo",38],["Leyton","Waterloo"]]

Output

[null,null,null,null,null,null,null,14.00000,11.00000,null,11.00000,null,12.00000]

Explanation

UndergroundSystem undergroundSystem = new UndergroundSystem();

undergroundSystem.checkIn(45, "Leyton", 3);

undergroundSystem.checkIn(32, "Paradise", 8);

undergroundSystem.checkIn(27, "Leyton", 10);

undergroundSystem.checkOut(45, "Waterloo", 15); // Customer 45 "Leyton" -> "Waterloo" in 15-3 = 12

undergroundSystem.checkOut(27, "Waterloo", 20); // Customer 27 "Leyton" -> "Waterloo" in 20-10 = 10

undergroundSystem.checkOut(32, "Cambridge", 22); // Customer 32 "Paradise" -> "Cambridge" in 22-8 = 14

undergroundSystem.getAverageTime("Paradise", "Cambridge"); // return 14.00000. One trip "Paradise" -> "Cambridge", (14) / 1 = 14

undergroundSystem.getAverageTime("Leyton", "Waterloo");   // return 11.00000. Two trips "Leyton" -> "Waterloo", (10 + 12) / 2 = 11

undergroundSystem.checkIn(10, "Leyton", 24);

undergroundSystem.getAverageTime("Leyton", "Waterloo");   // return 11.00000

undergroundSystem.checkOut(10, "Waterloo", 38); // Customer 10 "Leyton" -> "Waterloo" in 38-24 = 14

undergroundSystem.getAverageTime("Leyton", "Waterloo");   // return 12.00000. Three trips "Leyton" -> "Waterloo", (10 + 12 + 14) / 3 = 12

Example 2:

Input

["UndergroundSystem","checkIn","checkOut","getAverageTime","checkIn","checkOut","getAverageTime","checkIn","checkOut","getAverageTime"]

[[],[10,"Leyton",3],[10,"Paradise",8],["Leyton","Paradise"],[5,"Leyton",10],[5,"Paradise",16],["Leyton","Paradise"],[2,"Leyton",21],[2,"Paradise",30],["Leyton","Paradise"]]

Output

[null,null,null,5.00000,null,null,5.50000,null,null,6.66667]

Explanation

UndergroundSystem undergroundSystem = new UndergroundSystem();

undergroundSystem.checkIn(10, "Leyton", 3);

undergroundSystem.checkOut(10, "Paradise", 8); // Customer 10 "Leyton" -> "Paradise" in 8-3 = 5

undergroundSystem.getAverageTime("Leyton", "Paradise"); // return 5.00000, (5) / 1 = 5

undergroundSystem.checkIn(5, "Leyton", 10);

undergroundSystem.checkOut(5, "Paradise", 16); // Customer 5 "Leyton" -> "Paradise" in 16-10 = 6

undergroundSystem.getAverageTime("Leyton", "Paradise"); // return 5.50000, (5 + 6) / 2 = 5.5

undergroundSystem.checkIn(2, "Leyton", 21);

undergroundSystem.checkOut(2, "Paradise", 30); // Customer 2 "Leyton" -> "Paradise" in 30-21 = 9

undergroundSystem.getAverageTime("Leyton", "Paradise"); // return 6.66667, (5 + 6 + 9) / 3 = 6.66667

Constraints:

- 1 <= id, t <= 10^6
- 1 <= stationName.length, startStation.length, endStation.length <= 10
- All strings consist of uppercase and lowercase English letters and digits.
- There will be at most 2 * 10^4 calls **in total** to checkIn, checkOut, and getAverageTime.
- Answers within 10^{-5} of the actual value will be accepted.

CODE:

```
class UndergroundSystem {
    HashMap<Integer, String> station = new HashMap<>();
    HashMap<Integer, Integer> time = new HashMap<>();
    HashMap<String, Integer> total = new HashMap<>();
    HashMap<String, Integer> count = new HashMap<>();
    public UndergroundSystem() {
    }
    public void checkIn(int id, String stationName, int t) {
        station.put(id, stationName);
        time.put(id, t);
    }
```

```
    public void checkOut(int id, String stationName, int t) {
        String start = station.get(id);
        int startTime = time.get(id);
        String key = start + "->" + stationName;
        int travelTime = t - startTime;
        total.put(key, total.getOrDefault(key, 0) + travelTime);
        count.put(key, count.getOrDefault(key, 0) + 1);
        station.remove(id);
        time.remove(id);
    }
    public double getAverageTime(String startStation, String endStation) {
        String key = startStation + "->" + endStation;
        return (double) total.get(key) / count.get(key);
    }
}
```

INPUT:

["UndergroundSystem","checkIn","checkIn","checkIn","checkOut","checkOut","checkOut","getAverageTime","getAverageTime","checkIn","getAverageTime","checkOut","getAverageTime"]

[[],[45,"Leyton",3],[32,"Paradise",8],[27,"Leyton",10],[45,"Waterloo",15],[27,"Waterloo",20],[32,"Cambridge",22],["Paradise","Cambridge"],["Leyton","Waterloo"],[10,"Leyton",24],["Leyton","Waterloo"],[10,"Waterloo",38],["Leyton","Waterloo"]]

OUTPUT:

[null,null,null,null,null,null,null,14.00000,11.00000,null,11.00000,null,12.00000]

PROBLEM STATEMENT:

You have a **browser** of one tab where you start on the homepage and you can visit another url, get back in the history number of steps or move forward in the history number of steps.

Implement the BrowserHistory class:

- BrowserHistory(string homepage) Initializes the object with the homepage of the browser.

- void visit(string url) Visits url from the current page. It clears up all the forward history.
- string back(int steps) Move steps back in history. If you can only return x steps in the history and steps > x, you will return only x steps. Return the current url after moving back in history **at most** steps.
- string forward(int steps) Move steps forward in history. If you can only forward x steps in the history and steps > x, you will forward only x steps. Return the current url after forwarding in history **at most** steps.

Example:

Input:

["BrowserHistory","visit","visit","visit","back","back","forward","visit","forward","back","back"]

[["leetcode.com"],["google.com"],["facebook.com"],["youtube.com"],[1],[1],[1],["linkedin.com"],[2],[2],[7]]

Output:

[null,null,null,null,"facebook.com","google.com","facebook.com",null,"linkedin.com","google.com","leetcode.com"]

Explanation:

BrowserHistory browserHistory = new BrowserHistory("leetcode.com");

browserHistory.visit("google.com"); // You are in "leetcode.com". Visit "google.com"

browserHistory.visit("facebook.com"); // You are in "google.com". Visit "facebook.com"

browserHistory.visit("youtube.com"); // You are in "facebook.com". Visit "youtube.com"

browserHistory.back(1); // You are in "youtube.com", move back to "facebook.com" return "facebook.com"

browserHistory.back(1); // You are in "facebook.com", move back to "google.com" return "google.com"

browserHistory.forward(1); // You are in "google.com", move forward to "facebook.com" return "facebook.com"

browserHistory.visit("linkedin.com"); // You are in "facebook.com". Visit "linkedin.com"

browserHistory.forward(2); // You are in "linkedin.com", you cannot move forward any steps.

browserHistory.back(2); // You are in "linkedin.com", move back two steps to "facebook.com" then to "google.com". return "google.com"

browserHistory.back(7); // You are in "google.com", you can move back only one step to "leetcode.com". return "leetcode.com"

Constraints:

- 1 <= homepage.length <= 20
- 1 <= url.length <= 20
- 1 <= steps <= 100
- homepage and url consist of '.' or lower case English letters.
- At most 5000 calls will be made to visit, back, and forward.

CODE:

```
import java.util.*;
class BrowserHistory {
    ArrayList<String> history;
    int current;
    public BrowserHistory(String homepage) {
        history = new ArrayList<>();
        history.add(homepage);
        current = 0;
    }
    public void visit(String url) {
            while (history.size() > current + 1) {
          history.remove(history.size() - 1);
        }
        history.add(url);
        current++;
    }
```

```
    public String back(int steps) {
        current = Math.max(0, current - steps);
        return history.get(current);
    }
    public String forward(int steps) {
        current = Math.min(history.size() - 1, current + steps);
        return history.get(current);
    }
}
```

INPUT:

["BrowserHistory","visit","visit","visit","back","back","forward","visit","forward","back","back"]

[["leetcode.com"],["google.com"],["facebook.com"],["youtube.com"],[1],[1],[1],["linkedin.com"],[2],[2],[7]]

OUTPUT:

[null,null,null,null,"facebook.com","google.com","facebook.com",null,"linkedin.com","google.com","leetcode.com"]

PROBLEM STATEMENT:

Design a HashSet without using any built-in hash table libraries.

Implement MyHashSet class:

- void add(key) Inserts the value key into the HashSet.
- bool contains(key) Returns whether the value key exists in the HashSet or not.
- void remove(key) Removes the value key in the HashSet. If key does not exist in the HashSet, do nothing.

Example 1:

Input

["MyHashSet", "add", "add", "contains", "contains", "add", "contains", "remove", "contains"]

[[], [1], [2], [1], [3], [2], [2], [2], [2]]

Output

[null, null, null, true, false, null, true, null, false]

Explanation

MyHashSet myHashSet = new MyHashSet();

myHashSet.add(1);   // set = [1]

myHashSet.add(2);   // set = [1, 2]

myHashSet.contains(1); // return True

myHashSet.contains(3); // return False, (not found)

myHashSet.add(2);   // set = [1, 2]

myHashSet.contains(2); // return True

myHashSet.remove(2);   // set = [1]

myHashSet.contains(2); // return False, (already removed)

Constraints:

- $0 <= key <= 10^6$
- At most 10^4 calls will be made to add, remove, and contains.

CODE:

```
class MyHashSet {
    boolean[] set;
    public MyHashSet() {
        set = new boolean[1000001];
    }
    public void add(int key) {
        set[key] = true;
    }
    public void remove(int key) {
        set[key] = false;
    }
    public boolean contains(int key) {
        return set[key];
```

```
    }
}
```

INPUT:

["MyHashSet","add","add","add","remove","contains","add","add","add","remove","contains","add","add","add","remove","contains","add","add","add","remove","contains"]

[[],[1],[10001],[1],[1],[1],[7],[10007],[7],[7],[7],[123],[10123],[123],[123],[123],[5000],[15000],[5000],[5000],[5000]]

OUTPUT:

[null,null,null,null,null,false,null,null,null,null,false,null,null,null,null,false,null,null,null,null,false]

PROBLEM STATEMENT:

HackerLand University has the following grading policy:

Every student receives a in the inclusive range from 0 to 100 .

Any grade less than 40 is a failing grade.

Sam is a professor at the university and likes to round each student's grade according to these rules:

If the difference between the grade and the next multiple of 5 is less than 3, round grade up to the next multiple of 5.

If the value of grade is less than 38, no rounding occurs as the result will still be a failing grade.

Examples

grade=84 round to 85 (85 - 84 is less than 3)

 grade=29do not round (result is less than 38)

grade=57 do not round (60 - 57 is 3 or higher)

Given the initial value of grade for each of Sam's n students, write code to automate the rounding process.

Function Description

Complete the function gradingStudents with the following parameter(s):

int grades[n]: the grades before rounding

Returns

int[n]: the grades after rounding

Input Format

The first line contains a single integer, the number of students.

Each line i of the n subsequent lines contains a single integer,grades[i] .

Constraints

1<=n<=60

 0<=grades[i]<=100

Explanation 0

ID	Original Grade	Final Grade
1	73	75
2	67	67
3	38	40
4	33	33

Student 1 received a 73, and the next multiple of 5 from 73 is 75. Since 75-73<3, the student's grade is rounded to 75.

Student 2 received a 67, and the next multiple of 5 from 67 is 70. Since 70-67=3, the grade will not be modified and the student's final grade is 67 .

Student 3 received a 38, and the next multiple of 5 from 38 is 40. Since 40-38<3, the student's grade will be rounded to 40.

Student 4 received a grade below 33, so the grade will not be modified and the student's final grade is 33.

CODE:

```
import java.util.*;

public class Main {

    public static List<Integer> gradingStudents(List<Integer> grades) {

        List<Integer> result = new ArrayList<>();

        for (int grade : grades) {

            if (grade < 38) {

                result.add(grade);
```

```
            } else {
                int nextMultiple = ((grade / 5) + 1) * 5;
                if (nextMultiple - grade < 3) {
                    result.add(nextMultiple);
                } else {
                    result.add(grade);
                }
            }
        }
        return result;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        List<Integer> grades = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            grades.add(sc.nextInt());
        }
        List<Integer> result = gradingStudents(grades);
        for (int grade : result) {
            System.out.println(grade);
        }
    }
}
```

INPUT:

```
4
73
67
38
```

33

OUTPUT:

75

67

40

33

PROBLEM STATEMENT:

Using *inheritance*, one class can acquire the properties of others. Consider the following *Animal* class:

```
class Animal{
    void walk(){
        System.out.println("I am walking");
    }
}
```

This class has only one method, *walk*. Next, we want to create a *Bird* class that also has a *fly* method. We do this using *extends* keyword:

```
class Bird extends Animal {
    void fly() {
        System.out.println("I am flying");
    }
}
```

Finally, we can create a Bird object that can both *fly* and *walk*.

```
public class Solution{
   public static void main(String[] args){


      Bird bird = new Bird();
      bird.walk();
      bird.fly();
   }
}
```

The above code will print:

I am walking

I am flying

This means that a Bird object has all the properties that an Animal object has, as well as some additional unique properties.

The code above is provided for you in your editor. You must add a *sing* method to the *Bird* class, then modify the *main* method accordingly so that the code prints the following lines:

I am walking

I am flying

I am singing

CODE:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;


class Animal{
    void walk()
    {
        System.out.println("I am walking");
    }
}
class Bird extends Animal
{
    void fly()
    {
        System.out.println("I am flying");
    }
```

```
    void sing()
    {
        System.out.println("I am singing");
    }
}
public class Solution{
   public static void main(String args[]){
      Bird bird = new Bird();
      bird.walk();
      bird.fly();
      bird.sing();
   }
}
```

OUTPUT:

```
I am walking
I am flying
I am singing
```

PROBLEM STATEMENT:

Write the following code in your editor below:

1. A class named *Arithmetic* with a method named *add* that takes svg integers as parameters and returns an integer denoting their sum.
2. A class named *Adder* that inherits from a superclass named *Arithmetic*.

Your classes should not be be public.

Input Format

You are not responsible for reading any input from stdin; a locked code stub will test your submission by calling the *add* method on an *Adder* object and passing it 2 integer parameters.

Output Format

You are not responsible for printing anything to stdout. Your *add* method must return the sum of its parameters.

Sample Output

The *main* method in the *Solution* class above should print the following:

My superclass is: Arithmetic

42 13 20

CODE:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
class Arithmetic {
    int add(int a, int b) {
        return a + b;
    }
}
class Adder extends Arithmetic {
}
public class Solution{
    public static void main(String []args){
        Adder a = new Adder();
        System.out.println("My superclass is: " +
a.getClass().getSuperclass().getName());
        System.out.print(a.add(10,32) + " " + a.add(10,3) + " " + a.add(10,10) + "\n");
     }
}
```

OUTPUT:

My superclass is: Arithmetic

42 13 20

PROBLEM STATEMENT:

Generic methods are a very efficient way to handle multiple datatypes using a single method. This problem will test your knowledge on Java Generic methods.

Let's say you have an integer array and a string array. You have to write a **single** method *printArray* that can print all the elements of both arrays. The method should be able to accept both integer arrays or string arrays.

You are given code in the editor. Complete the code so that it prints the following lines:

1

2

3

Hello

World

Do not use method overloading because your answer will not be accepted.

CODE:

```
import java.util.*;

public class Solution {
    public static <T> void printArray(T[] array) {
        for (T element : array) {
            System.out.println(element);
        }
    }
    public static void main(String[] args) {
        Integer[] intArray = {1, 2, 3};
        String[] stringArray = {"Hello", "World"};
        printArray(intArray);
        printArray(stringArray);
    }
}
```

OUTPUT:

```
1
```

2

3

Hello

World

PROBLEM STATEMENT:

A kingdom consists of a king, his children, his grandchildren, and so on. Every once in a while, someone in the family dies or a child is born.
The kingdom has a well-defined order of inheritance that consists of the king as the first member. Let's define the recursive function Successor(x, curOrder), which given a person x and the inheritance order so far, returns who should be the next person after x in the order of inheritance.

Successor(x, curOrder):

if x has no children or all of x's children are in curOrder:

if x is the king return null

else return Successor(x's parent, curOrder)

else return x's oldest child who's not in curOrder

For example, assume we have a kingdom that consists of the king, his children Alice and Bob (Alice is older than Bob), and finally Alice's son Jack.

1. In the beginning, curOrder will be ["king"].
2. Calling Successor(king, curOrder) will return Alice, so we append to curOrder to get ["king", "Alice"].
3. Calling Successor(Alice, curOrder) will return Jack, so we append to curOrder to get ["king", "Alice", "Jack"].
4. Calling Successor(Jack, curOrder) will return Bob, so we append to curOrder to get ["king", "Alice", "Jack", "Bob"].
5. Calling Successor(Bob, curOrder) will return null. Thus the order of inheritance will be ["king", "Alice", "Jack", "Bob"].

Using the above function, we can always obtain a unique order of inheritance.
Implement the ThroneInheritance class:

- ThroneInheritance(string kingName) Initializes an object of the ThroneInheritance class. The name of the king is given as part of the constructor.
- void birth(string parentName, string childName) Indicates that parentName gave birth to childName.

- void death(string name) Indicates the death of name. The death of the person doesn't affect the Successor function nor the current inheritance order. You can treat it as just marking the person as dead.
- string[] getInheritanceOrder() Returns a list representing the current order of inheritance **excluding** dead people.

Example 1:

Input

["ThroneInheritance", "birth", "birth", "birth", "birth", "birth", "birth", "getInheritanceOrder", "death", "getInheritanceOrder"]

[["king"], ["king", "andy"], ["king", "bob"], ["king", "catherine"], ["andy", "matthew"], ["bob", "alex"], ["bob", "asha"], [null], ["bob"], [null]]

Output

[null, null, null, null, null, null, null, ["king", "andy", "matthew", "bob", "alex", "asha", "catherine"], null, ["king", "andy", "matthew", "alex", "asha", "catherine"]]

Explanation

ThroneInheritance t= new ThroneInheritance("king"); // order: king

t.birth("king", "andy"); // order: king > andy

t.birth("king", "bob"); // order: king > andy > bob

t.birth("king", "catherine"); // order: king > andy > bob > catherine

t.birth("andy", "matthew"); // order: king > andy > matthew > bob > catherine

t.birth("bob", "alex"); // order: king > andy > matthew > bob > alex > catherine

t.birth("bob", "asha"); // order: king > andy > matthew > bob > alex > asha > catherine

t.getInheritanceOrder(); // return ["king", "andy", "matthew", "bob", "alex", "asha", "catherine"]

t.death("bob"); // order: king > andy > matthew > bob > alex > asha > catherine

t.getInheritanceOrder(); // return ["king", "andy", "matthew", "alex", "asha", "catherine"]

Constraints:

- 1 <= kingName.length, parentName.length, childName.length, name.length <= 15

- kingName, parentName, childName, and name consist of lowercase English letters only.
- All arguments childName and kingName are **distinct**.
- All name arguments of death will be passed to either the constructor or as childName to birth first.
- For each call to birth(parentName, childName), it is guaranteed that parentName is alive.
- At most 105 calls will be made to birth and death.
- At most 10 calls will be made to getInheritanceOrder.

CODE:

```
import java.util.*;

class ThroneInheritance {
    private Map<String, List<String>> children;
    private Set<String> dead;
    private String king;
    public ThroneInheritance(String kingName) {
        king = kingName;
        children = new HashMap<>();
        dead = new HashSet<>();
        children.put(kingName, new ArrayList<>());
    }
    public void birth(String parentName, String childName) {
        children.get(parentName).add(childName);
        children.put(childName, new ArrayList<>());
    }
    public void death(String name) {
        dead.add(name);
    }
    public List<String> getInheritanceOrder() {
        List<String> order = new ArrayList<>();
```

```
        dfs(king, order);
        return order;
    }
    private void dfs(String person, List<String> order) {
        if (!dead.contains(person)) {
            order.add(person);
        }
        for (String child : children.get(person)) {
            dfs(child, order);
        }
    }
}
```

INPUT:

["ThroneInheritance","birth","birth","birth","birth","birth","birth","getInheritanceOrder","death","getInheritanceOrder"]

[["king"],["king","andy"],["king","bob"],["king","catherine"],["andy","matthew"],["bob","alex"],["bob","asha"],[null],["bob"],[null]]

OUTPUT:

[null,null,null,null,null,null,null,["king","andy","matthew","bob","alex","asha","catherine"],null,["king","andy","matthew","alex","asha","catherine"]]

WEEK 8

PROBLEM STATEMENT:

When a subclass inherits from a superclass, it also inherits its methods; however, it can also *override* the superclass methods (as well as declare and implement new ones). Consider the following *Sports* class:

```
class Sports{
    String getName(){
        return "Generic Sports";
    }
    void getNumberOfTeamMembers(){
```

```
        System.out.println( "Each team has n players in " + getName() );
    }
}
```

Next, we create a *Soccer* class that inherits from the *Sports* class. We can override the *getName* method and return a different, subclass-specific string:

```
class Soccer extends Sports{
    @Override
    String getName(){
        return "Soccer Class";
    }
}
```

Note: When overriding a method, you should precede it with the @Override annotation. The parameter(s) and return type of an overridden method must be exactly the same as those of the method inherited from the supertype.

Task

Complete the code in your editor by writing an overridden *getNumberOfTeamMembers* method that prints the same statement as the superclass' *getNumberOfTeamMembers* method, except that it replaces svg with svg (the number of players on a Soccer team).

Output Format

When executed, your completed code should print the following:

```
Generic Sports
Each team has n players in Generic Sports
Soccer Class
Each team has 11 players in Soccer Class
```

CODE:

```
import java.util.*;
class Sports{
    String getName(){
        return "Generic Sports";
```

```
    }
    void getNumberOfTeamMembers(){
        System.out.println( "Each team has n players in " + getName() );
    }
}
class Soccer extends Sports{
    @Override
    String getName(){
        return "Soccer Class";
    }
    @Override
void getNumberOfTeamMembers() {
    System.out.println("Each team has 11 players in " + getName());
}
    }
public class Solution{
    public static void main(String []args){
        Sports c1 = new Sports();
        Soccer c2 = new Soccer();
        System.out.println(c1.getName());
        c1.getNumberOfTeamMembers();
        System.out.println(c2.getName());
        c2.getNumberOfTeamMembers();
    }
}
```

OUTPUT:

```
Generic Sports
Each team has n players in Generic Sports
Soccer Class
```

Each team has 11 players in Soccer Class

PROBLEM STATEMENT:

When a method in a subclass overrides a method in superclass, it is still possible to call the overridden method using **super** keyword. If you write *super.func()* to call the function *func()*, it will call the method that was defined in the superclass.

You are given a partially completed code in the editor. Modify the code so that the code prints the following text:

Hello I am a motorcycle, I am a cycle with an engine.

My ancestor is a cycle who is a vehicle with pedals.

CODE:

```
import java.util.*;
import java.io.*;
class BiCycle{
    String define_me(){
        return "a vehicle with pedals.";
    }
}
class MotorCycle extends BiCycle{
    String define_me(){
        return "a cycle with an engine.";
    }
    MotorCycle(){
        System.out.println("Hello I am a motorcycle, I am "+ define_me());
        String temp=super.define_me();
        System.out.println("My ancestor is a cycle who is "+ temp );
    }
}
class Solution{
    public static void main(String []args){
        MotorCycle M=new MotorCycle();
```

```
    }
}
```

OUTPUT:

Hello I am a motorcycle, I am a cycle with an engine.

My ancestor is a cycle who is a vehicle with pedals.

PROBLEM STATEMENT:

Complete the *Singleton* class in your editor which contains the following components:

1. A *private Singleton* non parameterized constructor.
2. A *public* String instance variable named str .
3. Write a *static* method named *getSingleInstance* that returns the single instance of the *Singleton* class.

Once submitted, our hidden *Solution* class will check your code by taking a String as input and then using your *Singleton* class to print a line.

Input Format

You will not be handling any input in this challenge.

Output Format

You will not be producing any output in this challenge.

CODE:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
import java.lang.reflect.*;
class Singleton{
    private static Singleton singleInstance = new Singleton();
    public String str;
    private Singleton() {
    }
```

```
    public static Singleton getSingleInstance() {
        return singleInstance;
     }
}
public class Main {
 public static void main(String args[])throws Exception{
    Scanner sc=new Scanner(System.in);
    Singleton s1 = Singleton.getSingleInstance(); //retrive the single instance
    Singleton s2=Singleton.getSingleInstance();
    assert(s1==s2);
    Class c=s1.getClass();
    Constructor[] allConstructors = c.getDeclaredConstructors();
    assert allConstructors.length==1;
    for (Constructor ctor : allConstructors)
    {
        if(ctor.getModifiers()!=2 || !ctor.toString().equals("private Singleton()")) //The
constructor must be private
        {
            System.out.println("Wrong class!");
        }
    }
    String str=sc.nextLine();
    s1.str=str;
    s2.str=str;
    System.out.println("Hello I am a singleton! Let me say "+str+" to you");
 }
}
```

INPUT:

hello world

OUTPUT:

Hello I am a singleton! Let me say hello world to you

PROBLEM STATEMENT:

In this problem you must NOT generate any output on your own. Any such solution will be considered as being against the rules and its author will be disqualified. The output of your solution must be generated by the uneditable code provided for you in the solution template.

An important concept in Object-Oriented Programming is the open/closed principle, which means writing code that is open to *extension* but closed to *modification*. In other words, new functionality should be added by writing an extension for the existing code rather than modifying it and potentially breaking other code that uses it. This challenge simulates a real-life problem where the open/closed principle can and should be applied.

A *Tree* class implementing a rooted tree is provided in the editor. It has the following publicly available methods:

- getValue(): Returns the *value* stored in the node.
- getColor(): Returns the *color* of the node.
- getDepth(): Returns the depth of the node. Recall that the depth of a node is the number of edges between the node and the tree's root, so the tree's root has depth 0 and each descendant node's depth is equal to the depth of its parent node +1.

In this challenge, we treat the internal implementation of the tree as being closed to modification, so we cannot directly modify it; however, as with real-world situations, the implementation is written in such a way that it allows external classes to extend and build upon its functionality. More specifically, it allows objects of the *TreeVis* class (a Visitor Design Pattern) to visit the tree and traverse the tree structure via the accept method.

There are two parts to this challenge.

Part I: Implement Three Different Visitors

Each class has three methods you must write implementations for:

1. getResult(): Return an integer denoting the result, which is different for each class:
    - The *SumInLeavesVisitor* implementation must return the sum of the values in the tree's *leaves* only.

- The *ProductRedNodesVisitor* implementation must return the *product* of values stored in all *red* nodes, including leaves, computed modulo 10*9+7. Note that the product of zero values is equal to 1.
- The *FancyVisitor* implementation must return the absolute difference between the sum of values stored in the tree's *non-leaf nodes at even depth* and the sum of values stored in the tree's *green leaf nodes*. Recall that zero is an even number.

2. visitNode(TreeNode node): Implement the logic responsible for visiting the tree's *non-leaf* nodes such that the *getResult* method returns the correct result for the implementing class' visitor.
3. visitLeaf(TreeLeaf leaf): Implement the logic responsible for visiting the tree's *leaf* nodes such that the *getResult* method returns the correct result for the implementing class' visitor.

Part II: Read and Build the Tree

Read the svg-node tree, where each node is numbered from 1 to n. The tree is given as a list of node values (x1,x2,…,xn), a list of node colors (svg), and a list of edges. Construct this tree as an instance of the *Tree* class. The tree is always rooted at node number 1.

Your implementations of the three visitor classes will be tested on the tree you built from the given input.

Input Format

The first line contains a single integer, n, denoting the number of nodes in the tree.
The second line contains n space-separated integers describing the respective values of x1,x2,….xn.
The third line contains svg space-separated binary integers describing the respective values of c1,c2,….cn. Each ci denotes the color of the i*th node, where 0 denotes *red* and 1 denotes *green*.
Each of the n-1 subsequent lines contains two space-separated integers, svg and svg, describing an edge between nodes ui and vi.

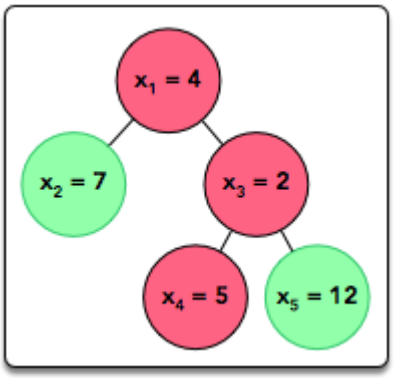
Constraints

- 2<=n<=10*5
- 1<=xi<=10*3
- Ci = {0,1}
- 1<=vi,ui<=n
- It is guaranteed that the tree is rooted at node 1.

Output Format

Do not print anything to stdout, as this is handled by locked stub code in the editor. The three getResult() methods provided for you must return an integer denoting the result for that class' visitor (defined above). Note that the value returned by *ProductRedNodesVisitor*'s *getResult* method must be computed modulo 10*9+7.

Explanation



Locked stub code in the editor tests your three class implementations as follows:

1. Creates a *SumInLeavesVisitor* object whose *getResult* method returns the *sum* of the *leaves* in the tree, which is 7+5+12=24. The locked stub code prints the returned value on a new line.
2. Creates a *ProductOfRedNodesVisitor* object whose *getResult* method returns the *product* of the *red* nodes, which is 4*2*5 = 40. The locked stub code prints the returned value on a new line.
3. Creates a *FancyVisitor* object whose *getResult* method returns the *absolute difference* between the *sum* of the values of *non-leaf* nodes at *even depth* and the *sum* of the values of *green leaf nodes*, which is |4-(7+12)| = 15. The locked stub code prints the returned value on a new line.

CODE:

```
import java.io.*;

import java.util.*;

enum Color {

    RED, GREEN

}

abstract class Tree {

    private int value;

    private Color color;
```

```
    private int depth;
    public Tree(int value, Color color, int depth) {
        this.value = value;
        this.color = color;
        this.depth = depth;
    }
    public int getValue() {
        return value;
    }
    public Color getColor() {
        return color;
    }
    public int getDepth() {
        return depth;
    }
    public abstract void accept(TreeVis visitor);
}
class TreeNode extends Tree {
    private ArrayList<Tree> children = new ArrayList<>();
    public TreeNode(int value, Color color, int depth) {
        super(value, color, depth);
    }
    public void accept(TreeVis visitor) {
        visitor.visitNode(this);
        for (Tree child : children) {
            child.accept(visitor);
        }
    }
    public void addChild(Tree child) {
```

```
        children.add(child);
    }
}
class TreeLeaf extends Tree {
    public TreeLeaf(int value, Color color, int depth) {
        super(value, color, depth);
    }
    public void accept(TreeVis visitor) {
        visitor.visitLeaf(this);
    }
}
abstract class TreeVis {
    public abstract int getResult();
    public abstract void visitNode(TreeNode node);
    public abstract void visitLeaf(TreeLeaf leaf);
}
class SumInLeavesVisitor extends TreeVis {
    private int sum = 0;
    public int getResult() {
        return sum;
    }
    public void visitNode(TreeNode node) {
    }
    public void visitLeaf(TreeLeaf leaf) {
        sum += leaf.getValue();
    }
}
class ProductOfRedNodesVisitor extends TreeVis {
    private static final long MOD = 1000000007;
```

```
    private long product = 1;

    public int getResult() {
        return (int) product;
    }
    public void visitNode(TreeNode node) {
        if (node.getColor() == Color.RED) {
            product = (product * node.getValue()) % MOD;
        }
    }
    public void visitLeaf(TreeLeaf leaf) {
        if (leaf.getColor() == Color.RED) {
            product = (product * leaf.getValue()) % MOD;
        }
    }
}
class FancyVisitor extends TreeVis {
    private int evenDepthSum = 0;
    private int greenLeafSum = 0;
    public int getResult() {
        return Math.abs(evenDepthSum - greenLeafSum);
    }
    public void visitNode(TreeNode node) {
        if (node.getDepth() % 2 == 0) {
            evenDepthSum += node.getValue();
        }
    }
    public void visitLeaf(TreeLeaf leaf) {
        if (leaf.getColor() == Color.GREEN) {
```

```
            greenLeafSum += leaf.getValue();
        }
    }
}
public class Solution {
    public static Tree solve() {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] values = new int[n];
        for (int i = 0; i < n; i++) {
            values[i] = sc.nextInt();
        }
        int[] colors = new int[n];
        for (int i = 0; i < n; i++) {
            colors[i] = sc.nextInt();
        }
        ArrayList<Integer>[] children = new ArrayList[n];
        for (int i = 0; i < n; i++) {
            children[i] = new ArrayList<>();
        }
        for (int i = 0; i < n - 1; i++) {
            int u = sc.nextInt() - 1;
            int v = sc.nextInt() - 1;
            children[u].add(v);
            children[v].add(u);
        }
        return buildTree(0, -1, 0, values, colors, children);
    }
    private static Tree buildTree(
```

```
    int node,
    int parent,
    int depth,
    int[] values,
    int[] colors,
    ArrayList<Integer>[] children) {
Color color = colors[node] == 0 ? Color.RED : Color.GREEN;
if (children[node].size() == 1 && node != 0) {
    return new TreeLeaf(
            values[node],
            color,
            depth
    );
}
TreeNode treeNode = new TreeNode(
        values[node],
        color,
        depth
);
for (int child : children[node]) {
    if (child != parent) {
        Tree childTree = buildTree(
                child,
                node,
                depth + 1,
                values,
                colors,
                children
        );
```

```
                treeNode.addChild(childTree);
            }
        }
        return treeNode;
    }
    public static void main(String[] args) {
        Tree root = solve();
        SumInLeavesVisitor vis1 = new SumInLeavesVisitor();
        ProductOfRedNodesVisitor vis2 = new ProductOfRedNodesVisitor();
        FancyVisitor vis3 = new FancyVisitor();
        root.accept(vis1);
        root.accept(vis2);
        root.accept(vis3);
        System.out.println(vis1.getResult());
        System.out.println(vis2.getResult());
        System.out.println(vis3.getResult());
    }
}
```

INPUT:

```
5
4 7 2 5 12
0 1 0 0 1
1 2
1 3
3 4
3 5
```

OUTPUT:

```
24
```

40

15

PROBLEM STATEMENT:

Java Iterator class can help you to iterate through every element in a collection. Here is a simple example:

```
import java.util.*;

public class Example{


    public static void main(String []args){
        ArrayList mylist = new ArrayList();
        mylist.add("Hello");
        mylist.add("Java");
        mylist.add("4");
        Iterator it = mylist.iterator();
        while(it.hasNext()){
            Object element = it.next();
            System.out.println((String)element);
        }
    }
}
```

In this problem you need to complete a method *func*. The method takes an *ArrayList* as input. In that *ArrayList* there is one or more integer numbers, then there is a special string "###", after that there are one or more other strings. A sample *ArrayList* may look like this:

element[0]=>42

element[1]=>10

element[2]=>"###"

element[3]=>"Hello"

element[4]=>"Java"

You have to modify the *func* method by editing at most 2 lines so that the code only prints the elements after the special string "###". For the sample above the output will be:

Hello

Java

Note: The stdin doesn't contain the string *"###"*, it is added in the *main* method.

To restore the original code in the editor, click the top left icon on the editor and create a new buffer.

CODE:

```
import java.util.*;

public class Main{
  static Iterator func(ArrayList mylist){
    Iterator it=mylist.iterator();
    while(it.hasNext()){
      Object element = it.next();
      if (element instanceof String)
        break;
     }
    return it;
  }
  @SuppressWarnings({ "unchecked" })
  public static void main(String []args){
    ArrayList mylist = new ArrayList();
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int m = sc.nextInt();
    for(int i = 0;i<n;i++){
      mylist.add(sc.nextInt());
    }
    mylist.add("###");
```

```
    for(int i=0;i<m;i++){
      mylist.add(sc.next());
    }
    Iterator it=func(mylist);
    while(it.hasNext()){
      Object element = it.next();
      System.out.println((String)element);
    }
  }
}
```

INPUT:

```
2 2
42
10
hello
java
```

OUTPUT:

```
hello
java
```

PROBLEM STATEMENT:

CODE:

INPUT:

OUTPUT: